

Variables, constantes et littéraux

Le support est propriété de **Bechir Bejaoui**

Ce chapitre explore les fondements de la manipulation des données en Python : les **variables** (noms qui réfèrent des valeurs), les **constantes** (valeurs conventionnellement immuables) et les **littéraux** (écriture directe des valeurs dans le code).

Vous découvrirez comment nommer correctement vos variables, affecter plusieurs valeurs à la fois, utiliser les différents types de littéraux (numériques, chaînes, booléens, collections), et mettre en pratique ces concepts à travers des exercices progressifs et des études de cas.

1. Les variables

1.1 Qu'est-ce qu'une variable ?

Une variable est un **nom** qui fait référence à un emplacement mémoire contenant une valeur. En Python, on crée une variable simplement en lui affectant une valeur avec le symbole `=`.

Contrairement à d'autres langages, Python n'exige pas de déclaration préalable du type : le type est **dynamiquement** déterminé par la valeur affectée.

```
x = 10          # x est de type int
x = "dix"       # x devient str (réaffectation possible)
```

1.2 Affectation multiple

Python permet d'affecter plusieurs variables en une seule ligne :

```
a, b, c = 1, 2.5, "trois"
print(a, b, c) # 1 2.5 trois
```

Une utilisation très pratique est l'**échange de valeurs** sans variable temporaire :

```
x, y = 5, 10
x, y = y, x
print(x, y) # 10 5
```

1.3 Règles de nommage

- Un nom de variable peut contenir des lettres (minuscules ou majuscules), des chiffres et le tiret bas `_`.
- Il doit **commencer par une lettre ou `_`** (pas par un chiffre).
- Sensible à la casse : `maVariable` $\neq$ `mavariabLe`.

- Ne pas utiliser les **mots-clés réservés** (if, for, while, def, class, import, etc.). Vous pouvez les lister avec `help('keywords')`.

Conventions (PEP 8) :

- Variables et fonctions : snake_case (ex. ma_variable).
- Classes : CamelCase (ex. MaClasse).
- Constantes : MAJUSCULES (ex. PI).

1.4 Représentation en mémoire

Représentation schématique :

Le nom de la variable pointe vers un emplacement mémoire qui contient la valeur.

Par exemple, après `x = 10`, on a :

`x` → [adresse mémoire] → 10

Quand on écrit `x = 10`, Python crée un objet entier 10 en mémoire et fait pointer le nom `x` vers cet objet. Si on réaffecte `x = 20`, `x` pointera vers un nouvel objet, l'ancien 10 sera éventuellement détruit par le ramasse-miettes.

1.5 Connaître le type d'une variable

La fonction `type()` renvoie le type de l'objet référencé.

```
age = 25
print(type(age))    # <class 'int'>
prix = 19.99
print(type(prix))   # <class 'float'>
```

2. Les constantes

Python ne possède pas de constante immuable au sens strict. Par convention, on utilise des noms en **majuscules** pour indiquer que la valeur ne doit pas être modifiée. C'est une convention que les développeurs respectent.

```
PI = 3.14159
GRAVITE = 9.8
```

Même si rien n'empêche d'écrire `PI = 3.14` plus tard, le nom en majuscule signale qu'il ne faut pas le faire.

3. Les littéraux

Un **littéral** est une notation directe d'une valeur dans le code source. Python supporte plusieurs familles de littéraux.

3.1 Littéraux numériques

Type	Préfixe	Exemple	Valeur décimale
Décimal	aucun	100	100
Binaire	0b	0b1010	10
Octal	0o	0o310	200
Hexadécimal	0x	0x12c	419
Flottant	aucun	3.14, 1.5e3	3.14, 1500.0
Complexe	j	2+3j	partie réelle 2, imaginaire 3

```
print(0b1010) # 10
print(0o310)  # 200
print(0x12c)  # 419
print(1.5e3)  # 1500.0
z = 2 + 3j
print(z.real, z.imag) # 2.0 3.0
```

3.2 Littéraux de chaînes

Type	Syntaxe	Exemple
Simple ou double	'...' ou "..."	"Bonjour"
Multiligne	"""...""" ou '''...'''	"""Ligne1\nLigne2"""
Brute (raw)	r"..."	r"C:\Users\Nom" (les \ ne sont pas interprétés)
Unicode (Python 3)	u"..." (optionnel)	u"Élève" (identique à "Élève")

```
s1 = "Simple"
s2 = 'Simple aussi'
s3 = """Ceci est une chaîne
sur plusieurs lignes."""
s4 = r"Le chemin est C:\temp" # Les \ ne sont pas des échappements
```

3.3 Littéraux booléens

Deux valeurs seulement : True et False. Elles sont de type bool.

```
est_vrai = True
est_faux = False
```

3.4 Littéral None

None représente l'absence de valeur. Il est souvent utilisé pour initialiser une variable ou comme valeur par défaut.

```
resultat = None
```

3.5 Littéraux de collections

Collection	Syntaxe	Exemple
Liste	[élément, ...]	[1, 2, 3]
Tuple	(élément, ...)	(1, 2, 3) ou 1, 2, 3
Dictionnaire	{clé: valeur, ...}	{"nom": "Alice", "age": 25}
Ensemble	{élément, ...}	{1, 2, 3}

```
fruits = ["pomme", "banane", "orange"]
coordonnees = (10.5, 20.3)
couleurs = {"rouge": "#FF0000", "vert": "#00FF00"}
voyelles = {'a', 'e', 'i', 'o', 'u', 'y'}
```

Attention : {} est un dictionnaire vide. Pour un ensemble vide, utilisez set().

4. Exercices progressifs

Exercice 1 : Premières affectations

1. Créez une variable `prenom` contenant votre prénom.
2. Créez une variable `age` contenant votre âge.
3. Affichez le type de chaque variable.
4. Réaffectez `age` avec la valeur "vingt ans". Quel est maintenant son type ?

```
prenom = "Marie"
age = 25
print(type(prenom)) # <class 'str'>
print(type(age))    # <class 'int'>
age = "vingt ans"
print(type(age))    # <class 'str'>
```

Exercice 2 : Échange de valeurs

Demandez deux nombres à l'utilisateur, stockez-les dans `a` et `b`, puis échangez leurs valeurs et affichez-les.

```
a = input("Entrez a : ")
b = input("Entrez b : ")
a, b = b, a
print("Après échange : a =", a, "b =", b)
```

Exercice 3 : Littéraux numériques

Convertissez les nombres suivants en décimal (à l'aide de `print`) :

- 0b1101
- 0o123
- 0x1A3

```
print(0b1101) # 13
print(0o123)  # 83
print(0x1A3)  # 419
```

Exercice 4 : Chaînes brutes

Affichez le chemin C:\Users\Public\Documents sans que les \ ne soient interprétés comme des échappements.

```
print(r"C:\Users\Public\Documents")
```

Exercice 5 : Collection simple

Créez une liste contenant trois nombres, un tuple contenant deux chaînes, un dictionnaire associant trois pays à leur capitale, et un ensemble de trois couleurs. Affichez chaque collection.

```
liste_nb = [1, 2, 3]
tuple_str = ("un", "deux")
capitale = {"France": "Paris", "Italie": "Rome", "Espagne": "Madrid"}
couleurs = {"rouge", "vert", "bleu"}
print(liste_nb, tuple_str, capitale, couleurs)
```

5. Études de cas (complexité progressive)

Cas 1 : Calcul de la circonférence et de l'aire d'un cercle

Niveau 1 : Utilisez une constante `PI` et demandez le rayon. Calculez et affichez la circonférence et l'aire.

```
PI = 3.14159
rayon = float(input("Rayon du cercle : "))
circonference = 2 * PI * rayon
aire = PI * rayon ** 2
print(f"Circonférence : {circonference:.2f}")
print(f"Aire : {aire:.2f}")
```

Niveau 2 : Ajoutez un menu pour choisir entre calculer la circonférence ou l'aire, ou les deux.

```
PI = 3.14159
rayon = float(input("Rayon : "))
print("1. Circonférence")
print("2. Aire")
print("3. Les deux")
choix = int(input("Votre choix : "))
if choix == 1:
    print(f"Circonférence : {2*PI*rayon:.2f}")
elif choix == 2:
    print(f"Aire : {PI*rayon**2:.2f}")
elif choix == 3:
    print(f"Circonférence : {2*PI*rayon:.2f}")
```

```

    print(f"Aire : {PI*rayon**2:.2f}")
else:
    print("Choix invalide")

```

Niveau 3 : Validez que le rayon est positif. Bouclez pour permettre plusieurs calculs.

```

PI = 3.14159
while True:
    saisie = input("Rayon (ou 'q' pour quitter) : ")
    if saisie.lower() == 'q':
        break
    try:
        rayon = float(saisie)
        if rayon <= 0:
            print("Le rayon doit être positif.")
            continue
        print(f"Circonférence : {2*PI*rayon:.2f}")
        print(f"Aire : {PI*rayon**2:.2f}")
    except ValueError:
        print("Saisie invalide.")

```

Cas 2 : Convertisseur de bases

Niveau 1 : Demandez un nombre décimal et affichez ses représentations binaire, octale et hexadécimale (avec les fonctions `bin()`, `oct()`, `hex()`).

```

n = int(input("Nombre décimal : "))
print(f"Binaire : {bin(n)}")
print(f"Octal : {oct(n)}")
print(f"Hexadécimal : {hex(n)}")

```

Niveau 2 : Proposez un menu inverse : l'utilisateur entre un nombre dans une base (binaire, octal, hexa) et vous le convertissez en décimal.

```

print("1. Binaire -> décimal")
print("2. Octal -> décimal")
print("3. Hexadécimal -> décimal")
choix = input("Choix : ")
valeur = input("Entrez le nombre : ")
if choix == '1':
    decimal = int(valeur, 2)
elif choix == '2':
    decimal = int(valeur, 8)
elif choix == '3':
    decimal = int(valeur, 16)
else:
    decimal = None
if decimal is not None:
    print("Valeur décimale :", decimal)
else:
    print("Choix invalide")

```

Niveau 3 : Permettez à l'utilisateur de choisir la base de départ et la base d'arrivée (2, 8, 10, 16). Gérez les erreurs de saisie.

```

def convertir(valeur, base_depart, base_arrivee):
    # Convertir d'abord en décimal
    try:
        decimal = int(valeur, base_depart)
    except ValueError:
        return None
    # Convertir du décimal vers la base d'arrivée

```

```

if base_arrivee == 2:
    return bin(decimal)
elif base_arrivee == 8:
    return oct(decimal)
elif base_arrivee == 10:
    return str(decimal)
elif base_arrivee == 16:
    return hex(decimal)
else:
    return None

print("Bases disponibles : 2, 8, 10, 16")
bd = int(input("Base de départ : "))
ba = int(input("Base d'arrivée : "))
val = input("Valeur à convertir : ")
resultat = convertir(val, bd, ba)
if resultat:
    print("Résultat :", resultat)
else:
    print("Erreur de conversion.")

```

Cas 3 : Gestion d'une petite base de données d'étudiants

Niveau 1 : Créez un dictionnaire où chaque clé est un nom d'étudiant et la valeur est son âge. Ajoutez trois étudiants. Affichez le dictionnaire.

```

etudiants = {"Alice": 22, "Bob": 24, "Charlie": 21}
print(etudiants)

```

Niveau 2 : Ajoutez une fonction pour calculer l'âge moyen. Utilisez les méthodes `values()` et `sum()`.

```

etudiants = {"Alice": 22, "Bob": 24, "Charlie": 21}
ages = list(etudiants.values())
moyenne = sum(ages) / len(ages)
print(f"Âge moyen : {moyenne:.1f}")

```

Niveau 3 : Permettez à l'utilisateur d'ajouter un nouvel étudiant, de modifier l'âge, de supprimer un étudiant. Sauvegardez les données dans un fichier (optionnel).

```

etudiants = {"Alice": 22, "Bob": 24, "Charlie": 21}
while True:
    print("\n1. Afficher tous")
    print("2. Ajouter/modifier")
    print("3. Supprimer")
    print("4. Moyenne d'âge")
    print("5. Quitter")
    choix = input("Choix : ")
    if choix == '1':
        for nom, age in etudiants.items():
            print(f"{nom} : {age} ans")
    elif choix == '2':
        nom = input("Nom : ")
        age = int(input("Âge : "))
        etudiants[nom] = age
    elif choix == '3':
        nom = input("Nom à supprimer : ")
        if nom in etudiants:
            del etudiants[nom]
            print("Supprimé.")
        else:
            print("Inconnu.")

```

```

elif choix == '4':
    if etudiants:
        moyenne = sum(etudiants.values()) / len(etudiants)
        print(f"Moyenne : {moyenne:.1f}")
    else:
        print("Aucun étudiant.")
elif choix == '5':
    break

```

Cas 4 : Analyse d'une phrase

Niveau 1 : Demandez une phrase et affichez le nombre de caractères (espaces compris) et le nombre de mots (avec `split()`).

```

phrase = input("Phrase : ")
print("Caractères :", len(phrase))
print("Mots :", len(phrase.split()))

```

Niveau 2 : Comptez le nombre de voyelles et de consonnes (lettres uniquement). Ignorez les autres caractères.

```

phrase = input("Phrase : ").lower()
voyelles = "aeiouy"
nb_voyelles = 0
nb_consonnes = 0
for c in phrase:
    if c.isalpha():
        if c in voyelles:
            nb_voyelles += 1
        else:
            nb_consonnes += 1
print(f"Voyelles : {nb_voyelles}, Consonnes : {nb_consonnes}")

```

Niveau 3 : En plus, affichez la phrase inversée, et la phrase en majuscules et minuscules.

```

phrase = input("Phrase : ")
print("Inversée :", phrase[::-1])
print("Majuscules :", phrase.upper())
print("Minuscules :", phrase.lower())

```

6. Schémas pour visualiser les concepts

6.1 Cycle de vie d'une variable

Représentation textuelle du déroulement :

1. Programme : `x = 10` → Mémoire : Crée un objet entier 10 et associe le nom `x`.
2. Programme : `x = 20` → Mémoire : Crée un objet entier 20, fait pointer `x` vers lui, et l'objet 10 est libéré (ramasse-miettes).

6.2 Les différents littéraux

Arborescence des littéraux Python :

- Numériques
 - Entier : 100
 - Binaire : 0b1010
 - Octal : 0o310
 - Hexa : 0x12c
 - Flottant : 3.14
 - Complexe : 2+3j
- Chaînes
 - Simple : 'Bonjour'
 - Double : "Hello"
 - Multiligne : """..."""
 - Brute : r"C:\temp"
- Booléens : True, False
- Absence : None
- Collections
 - Liste : [1,2,3]
 - Tuple : (1,2,3)
 - Dictionnaire : {"a":1}
 - Ensemble : {1,2,3}

6.3 Affectation multiple et échange

Avant échange :

- $a \rightarrow 5$
- $b \rightarrow 10$

Après échange :

- $a \rightarrow 10$
- $b \rightarrow 5$

7. Exercices de synthèse (plus complexes)

Exercice 11 : Générateur de mots de passe aléatoires

Utilisez le module `random` et les littéraux de chaînes pour générer un mot de passe de longueur 12 contenant au moins une minuscule, une majuscule, un chiffre et un caractère spécial.

```
import random
import string
```

```

minuscules = string.ascii_lowercase
majuscules = string.ascii_uppercase
chiffres = string.digits
speciaux = "!@#$$%^&*"
tous = minuscules + majuscules + chiffres + speciaux

mdp = [
    random.choice(minuscules),
    random.choice(majuscules),
    random.choice(chiffres),
    random.choice(speciaux)
]
mdp += random.choices(tous, k=8)
random.shuffle(mdp)
mdp_str = ''.join(mdp)
print(mdp_str)

```

Exercice 12 : Mini-système de notation

1. Demandez le nombre d'étudiants.
2. Pour chaque étudiant, demandez son nom et trois notes.
3. Stockez les informations dans un dictionnaire : clé = nom, valeur = liste des notes.
4. Calculez la moyenne de chaque étudiant.
5. Affichez le classement (ordre décroissant de moyenne).

```

n = int(input("Nombre d'étudiants : "))
etudiants = {}
for i in range(n):
    nom = input(f"Nom de l'étudiant {i+1} : ")
    notes = []
    for j in range(3):
        note = float(input(f>Note {j+1} : "))
        notes.append(note)
    etudiants[nom] = notes

# Calcul des moyennes et classement
moyennes = {nom: sum(notes)/len(notes) for nom, notes in etudiants.items()}
classement = sorted(moyennes.items(), key=lambda x: x[1], reverse=True)
print("\nClassement :")
for i, (nom, moy) in enumerate(classement, 1):
    print(f"{i}. {nom} : {moy:.2f}")

```

Exercice 13 : Gestion d'un stock (avec dictionnaire et listes)

1. Créez un dictionnaire stock où chaque clé est un produit (chaîne) et la valeur est la quantité (entier).
2. Implémentez un menu :
 - Afficher le stock
 - Ajouter un produit (ou augmenter la quantité)
 - Enlever un produit (ou diminuer)
 - Quitter

3. Sauvegardez le stock dans un fichier texte (optionnel).

```
stock = {}
while True:
    print("\n1. Afficher stock")
    print("2. Ajouter / augmenter")
    print("3. Retirer / diminuer")
    print("4. Quitter")
    choix = input("Choix : ")
    if choix == '1':
        if not stock:
            print("Stock vide.")
        else:
            for p, q in stock.items():
                print(f"{p} : {q}")
    elif choix == '2':
        produit = input("Produit : ")
        qte = int(input("Quantité à ajouter : "))
        stock[produit] = stock.get(produit, 0) + qte
    elif choix == '3':
        produit = input("Produit : ")
        if produit in stock:
            qte = int(input("Quantité à retirer : "))
            if qte >= stock[produit]:
                del stock[produit]
            else:
                stock[produit] -= qte
        else:
            print("Produit inconnu.")
    elif choix == '4':
        break
```

8. Ce qu'il faut retenir

- **Variable** : nom qui référence une valeur (type dynamique).
- **Affectation multiple** : pratique pour échanger des valeurs.
- **Conventions de nommage** : snake_case pour variables/fonctions, MAJUSCULES pour constantes.
- **Littéraux** : écriture directe des valeurs (numériques, chaînes, booléens, None, collections).
- Les chaînes brutes (r"...") évitent l'interprétation des \.
- Les collections (listes, tuples, dictionnaires, ensembles) sont des littéraux utiles.

Astuce : Utilisez `type()` pour vérifier le type d'une variable, surtout quand vous débutez.

9. Annexe : liste des mots-clés réservés

```
import keyword
print(keyword.kwlist)
# ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue',
'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in',
```

`'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']`

Évitez d'utiliser ces noms pour vos variables.

Avec ces bases solides, vous êtes maintenant prêt à aborder les structures de contrôle (conditions, boucles) et les fonctions. Bonne continuation !

20 Questions Intelligentes

Variables, Constantes et Littéraux — Niveau Intermédiaire

1) Une variable contient-elle une valeur ou une référence ?

Énoncé

Quand on écrit `x = 10`, que contient réellement `x` ?

Solution

"""

Explication :

En Python, une variable ne contient pas directement la valeur.

Elle référence un objet en mémoire.

"""

```
x = 10
```

```
y = x
```

```
print("x =", x)
```

```
print("y =", y)
```

```
print("Même objet ?", x is y)
```

Démonstration observable :

`x` et `y` pointent vers le même objet int tant que l'objet est immuable et partagé.

2) Réaffectation vs modification : quelle différence ?

Énoncé

Pourquoi une liste modifiée impacte plusieurs variables alors qu'un entier ne le fait pas ?

Solution

```
"""
Explication :
- Les entiers sont immuables.
- Les listes sont mutables.
Modifier un objet mutable modifie l'objet partagé.
"""
```

```
a = 10
b = a
a = 20
```

```
print("a =", a)
print("b =", b)
```

```
liste1 = [1, 2, 3]
liste2 = liste1
```

```
liste1.append(4)
```

```
print("liste1 =", liste1)
print("liste2 =", liste2)
```

Observation :

La liste est partagée, l'entier ne l'est plus après réaffectation.

3) Pourquoi id() change-t-il après une réaffectation ?

Énoncé

Pourquoi id(x) change-t-il après x = nouvelle_valeur ?

Solution

```
"""
Explication :
id() retourne l'adresse mémoire de l'objet.
"""
```

Une réaffectation crée un nouvel objet.

```
"""
```

```
x = 100  
print("id initial :", id(x))
```

```
x = 200  
print("id après réaffectation :", id(x))
```

Démonstration :

Deux objets distincts sont créés.

4) Pourquoi True + True vaut 2 ?

Énoncé

Pourquoi l'addition de booléens donne un entier ?

Solution

```
"""
```

```
Explication :  
bool est une sous-classe de int.  
True vaut 1, False vaut 0.  
"""
```

```
print(True + True)  
print(True * 5)  
print(isinstance(True, int))
```

Résultat observable :

True se comporte comme 1.

5) Différence entre is et == ?

Énoncé

Quand utiliser is plutôt que == ?

Solution

```
"""
```

```
Explication :  
== compare les valeurs.  
is compare les identités (même objet mémoire).  
"""
```

```
a = [1, 2]
b = [1, 2]

print("a == b :", a == b)
print("a is b :", a is b)

c = None
print("Comparer à None avec is :", c is None)
Conclusion :
is est recommandé pour None.
```

6) Pourquoi {} n'est pas un set vide ?

Énoncé

Pourquoi {} crée un dictionnaire et non un ensemble ?

Solution

```
"""
Explication :
{} est interprété comme un dictionnaire vide.
Un set vide nécessite set().
"""
```

```
d = {}
s = set()

print(type(d))
print(type(s))
```

7) Pourquoi une virgule crée un tuple ?

Énoncé

Pourquoi (5) n'est pas un tuple mais (5,) l'est ?

Solution

```
"""
Explication :
C'est la virgule qui définit le tuple.
"""
```

```
a = (5)
b = (5,)

print(type(a))
print(type(b))
```

8) Que se passe-t-il si on modifie une constante ?

Énoncé

Les constantes sont-elles réellement protégées ?

Solution

```
"""
Explication :
Les constantes sont une convention.
Python n'empêche pas leur modification.
"""
```

```
PI = 3.14159
print("PI initial :", PI)
```

```
PI = 3
print("PI modifié :", PI)
```

9) Différence entre chaîne brute et chaîne normale ?

Énoncé

Pourquoi r"C:\temp" est utile ?

Solution

```
"""
Explication :
Les chaînes normales interprètent les séquences d'échappement.
Les raw strings les ignorent.
"""
```



```
normal = "C:\\temp"
brute = r"C:\\temp"

print(normal)
print(brute)
```

10) Pourquoi $0.1 + 0.2 \neq 0.3$?

Énoncé

Pourquoi la comparaison échoue ?

Solution

```
"""
Explication :
Les flottants utilisent une représentation binaire approximative.
Cela entraîne des erreurs d'arrondi.
"""

print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)
Observation :
Utiliser round() ou math.isclose() en production.
```

11) Pourquoi une variable globale peut poser problème ?

Énoncé

Quels risques avec une variable modifiée globalement ?

Solution

```
"""
Explication :
Les variables globales rendent le code moins prévisible.
Elles augmentent le couplage.
"""

compteur = 0

def incrementer():
```

```
global compteur
compteur += 1

incrémenter()
print(compteur)
```

12) Pourquoi input() retourne toujours une chaîne ?

Énoncé

Pourquoi faut-il convertir après input() ?

Solution

```
"""
Explication :
input() retourne toujours une chaîne.
Il faut convertir vers int ou float si nécessaire.
"""

age = input("Age : ")
print(type(age))

age = int(age)
print(type(age))
```

13) Pourquoi None n'est pas False ?

Énoncé

Pourquoi None == False est faux ?

Solution

```
"""
Explication :
None représente l'absence de valeur.
False représente une valeur booléenne.
"""

print(None == False)
print(None is None)
```

14) Pourquoi une liste copiée peut modifier l'original ?

Énoncé

Quelle différence entre copie simple et copie indépendante ?

Solution

```
"""
Explication :
Affecter une liste copie la référence.
Pour une copie indépendante : list() ou slicing.
"""
```

```
a = [1, 2, 3]
b = a
c = a[:]

a.append(4)

print("a =", a)
print("b =", b)
print("c =", c)
```

15) Pourquoi les chaînes sont immuables ?

Énoncé

Pourquoi `s[0] = "A"` provoque une erreur ?

Solution

```
"""
Explication :
Les chaînes sont immuables.
Toute modification crée une nouvelle chaîne.
"""

s = "python"

try:
    s[0] = "P"
except TypeError as e:
    print("Erreur :", e)
```

16) Pourquoi utiliser enumerate plutôt qu'un compteur manuel ?

Énoncé

Quel avantage avec enumerate ?

Solution

```
"""
Explication :
enumerate fournit index et valeur.
Évite la gestion manuelle d'un compteur.
"""
```

```
liste = ["a", "b", "c"]
```

```
for index, valeur in enumerate(liste):
    print(index, valeur)
```

17) Pourquoi préférer f-string ?

Énoncé

Pourquoi f"{}" est plus moderne que concaténation ?

Solution

```
"""
Explication :
Les f-strings sont plus lisibles et performantes.
Elles permettent le formatage intégré.
"""
```

```
nom = "Ali"
```

```
age = 30
```

```
print(f"{nom} a {age} ans")
```

18) Pourquoi set supprime les doublons ?

Énoncé

Comment un set élimine les répétitions ?

Solution

```
"""
Explication :
Les ensembles ne permettent pas de doublons.
Ils utilisent un mécanisme de hachage.
"""
```

```
liste = [1, 1, 2, 3, 3, 4]
unique = set(liste)

print(unique)
```

19) Pourquoi les dictionnaires conservent l'ordre ?

Énoncé

Depuis quelle version l'ordre est-il garanti ?

Solution

```
"""
Explication :
Depuis Python 3.7, l'ordre d'insertion est garanti.
"""
```

```
d = {"a": 1, "b": 2, "c": 3}
print(d)
```

20) Pourquoi utiliser isinstance plutôt que type ?

Énoncé

Quelle différence conceptuelle ?

Solution

```
"""
Explication :
type(obj) teste le type exact.
isinstance accepte l'héritage.
Plus flexible et recommandé.
"""
```

```
class A:
    pass
```

```
class B(A):
    pass
```

```
b = B()
```

```
print(type(b) == A)
print(isinstance(b, A))
```